

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..25

CUSTOM OVERRIDES

 Copy as Markdown

Custom authentication overrides

The `serverpod_auth_idp` module covers most authentication needs, but you can replace it when you have requirements it does not meet. A custom override means two things: a server-side handler that turns a token into a signed-in user, and an app-side provider that sends that token with every request. This page covers both.

Server setup

With a custom override you decide how users are stored and how tokens are issued and validated.

Token validation

The token validation is performed by providing a custom `AuthenticationHandler` callback when initializing Serverpod. The callback should return an `AuthenticationInfo` object if the token is valid, otherwise `null`.

```
final pod = Serverpod(  
  args,  
  authenticationHandler: (Session session, String token) async {  
    /// Custom validation handler  
    if (token != 'valid') return null;  
  
    return AuthenticationInfo('1', <Scope>{}, authId: 'device-1');  
  },  
);
```

In the above example, the `authenticationHandler` callback is overridden with a custom validation method. The method returns an `AuthenticationInfo` object with `userIdentifier` "1" and no scopes if the token is the literal "valid", otherwise `null`.

The `AuthenticationInfo` constructor takes the user's identifier as a `String` (an empty string). The required `authId` field links the authentication to a specific device or token, so it can be revoked.



Ask AI

individually later.

NOTE

Inside the `authenticationHandler` callback, the `authenticated` field on the session is always `null`, since it is the handler itself that figures out who the user is.

Scopes

The scopes returned from the `authenticationHandler` are used to grant access to [scope-restricted endpoints](#). The `Scope` class is a simple wrapper around a nullable `String`, so you can format your scopes however you want.

A JWT (JSON Web Token) implementation would normally store the scopes inside the token. After extracting them, convert each string into a `Scope` object by calling the constructor:

```
List<String> scopes = extractScopes(token);
Set<Scope> userScopes = scopes.map((scope) => Scope(scope)).toSet();
```

Handling revoked authentication

When a user's authentication is revoked, the server must be told so it can act on the change, for example by closing method streams. Call `session.messages.authenticationRevoked` with the message type that matches the extent of the revocation.

```
var userIdentifier = '1';
var revokedScopes = ['write'];
var message = RevokedAuthenticationScope(
    scopes: revokedScopes,
);

await session.messages.authenticationRevoked(
    userIdentifier,
    message,
);
```

Parameters

- `userIdentifier` - The `userIdentifier` of the `AuthenticationInfo` object to be revoked.
- `message` - The revoked authentication event message. See below for the different type of messages.

Revoked authentication messages

There are three types of `RevokedAuthentication` messages that are used to specify the extent of the authentication revocation:

Message type	Description
<code>RevokedAuthenticationUser</code>	All authentication is revoked for a user.
<code>RevokedAuthenticationAuthId</code>	A single authentication id is revoked for the user. This should match the <code>authId</code> field in the <code>AuthenticationInfo</code> object.
<code>RevokedAuthenticationScope</code>	List of scopes that have been revoked for a user.

Send the token to the app

You are responsible for implementing the endpoints that authenticate the user. The example below shows the shape of such an endpoint. The `authenticateUser` and `issueMyToken` functions are placeholders for your own logic.

```
class UserEndpoint extends Endpoint {
  Future<LoginResponse?> login(
    Session session,
    String username,
    String password,
  ) async {
    var identifier = await authenticateUser(session, username, password);
    if (identifier == null) return null;

    return issueMyToken(identifier, scopes: {});
  }
}
```

In the above example, the `login` method authenticates the user and creates an auth token. The token is then returned to the app.

Client setup

To authenticate from your app, configure an auth key provider on the client. The client then asks the provider for an authentication header value and includes it in every request to the server.

Configure auth key provider

Auth key providers need to implement the `ClientAuthKeyProvider` interface. The provider is assigned to the client's `authKeyProvider` field after construction, as the example below does. If no provider is configured, the client will not include authentication headers in requests to the server.

```
import 'package:serverpod_client/serverpod_client.dart';

class SimpleAuthKeyProvider implements ClientAuthKeyProvider {
  String? _key;

  @override
  Future<String?> get authHeaderValue async {
    if (_key == null) return null;
    return wrapAsBearerAuthHeaderValue(_key!);
  }

  Future<void> put(String key) async {
    _key = key;
  }

  Future<void> remove() async {
    _key = null;
  }
}

var client = Client('http://localhost:8080/')
  ..authKeyProvider = SimpleAuthKeyProvider()
  ..connectivityMonitor = FlutterConnectivityMonitor();
```

In the above example, the `SimpleAuthKeyProvider` is configured as the client's auth key provider. The `SimpleAuthKeyProvider` stores the token in memory and wraps it as a Bearer auth header value using the `wrapAsBearerAuthHeaderValue` utility function.

INFO

The `SimpleAuthKeyProvider` is not practical and should only be used for testing. A secure implementation of the auth key provider is available in the `serverpod_auth_core_flutter` package. It provides safe, persistent storage for the auth token.

Your app is responsible for storing the token in the auth key provider. Reach the provider through the client's `authKeyProvider` field, which you have to cast back to your own type. A getter saves you from repeating the cast:

```
SimpleAuthKeyProvider get authProvider =>
  client.authKeyProvider as SimpleAuthKeyProvider;
```

Authentication schemes

The token travels from the app to the server in the HTTP `authorization` header. Anyone who reads that header can use the token, so serve your production API over HTTPS. See [Security and TLS](#).

Serverpod does not pick a scheme for you. Whatever your auth key provider returns from `authHeaderValue` is sent as the header value.

The server accepts three schemes: `Bearer`, `Basic`, and `Digest`. The scheme name is matched case-sensitively, and a header in any other scheme is rejected with a 400 response before your `AuthenticationHandler` runs. Set `validateHeaders` to `false` in your server configuration to turn that check off and receive the raw header value instead.

Before calling your handler, the server unwraps the value. A `Bearer` value has its scheme prefix stripped, and a `Basic` value is base64-decoded, so your handler receives the plain token either way.

Use `Bearer` for a token with no internal structure, which is what the built-in authentication module does and what the example above shows. The `Basic` scheme is only suitable for a key shaped like `username:password`, because it is rejected when the decoded value has no colon or either part is empty.

Implementing your own authentication scheme

Return the header value you want from the `authHeaderValue` getter of your `ClientAuthKeyProvider` implementation. The utility functions `wrapAsBearerAuthHeaderValue` and `wrapAsBasicAuthHeaderValue` format the token for the two common schemes.

The header value must be compliant with the HTTP header format defined in RFC 9110 HTTP Semantics, 11.6.2. Authorization. See:

- [HTTP Authorization header](#)
- [RFC 9110, 11.6.2. Authorization](#)

The example below adapts the earlier code to bearer tokens, the scheme OAuth2 uses.

Client side:

```
import 'package:serverpod_client/serverpod_client.dart';

class MyOAuthKeyProvider implements ClientAuthKeyProvider {
  String? _key;

  @override
  Future<String?> get authHeaderValue async {
    if (_key == null) return null;
    return wrapAsBearerAuthHeaderValue(myBearerTokenObtainer(_key!));
  }

  Future<void> put(String key) async {
    _key = key;
  }
}
```

```

    }

    Future<void> remove() async {
      _key = null;
    }
  }

  var client = Client('http://localhost:8080/')
    ..authKeyProvider = MyOAuthKeyProvider()
    ..connectivityMonitor = FlutterConnectivityMonitor();

```

Server side:

```

final pod = Serverpod(
  args,
  authenticationHandler: (Session session, String token) async {
    /// Bearer token validation handler
    var (uid, scopes) = myBearerTokenValidator(token);
    if (uid == null) return null;

    return AuthenticationInfo(uid, scopes, authId: token);
  },
);

```

Related

- [Setup](#): the built-in authentication module, which these overrides replace.
- [The basics](#): how `requireLogin` and scopes restrict endpoints.
- [Token managers](#): the built-in token machinery.